maestro-adapter

Maestro Adapter

The Maestro adapter implements `MobileAdapter` using the Maestro CLI, a YAML-driven mobile testing framework. Each mobile action generates a YAML flow file in a temporary directory and invokes `maestro test` to execute it against a connected device or emulator.

Architecture

```
Go Challenge  -->  MaestroCLIAdapter  -->  Generate YAML flow file
                                                    |
                                           maestro test <flow.yaml>
                                                    |
                                           Maestro Engine
                                                    |
                                      Device / Emulator (Android or
iOS)
```

Maestro is a declarative mobile testing tool that uses YAML files to describe test flows. The adapter translates each `MobileAdapter` method call into a single-command YAML flow, writes it to a temporary file, and executes it via the `maestro` CLI.

YAML Flow Generation

The `MaestroFlow` struct defines a flow with an app ID and a list of commands. The `toYAML()` method serializes it into Maestro-compatible YAML:

```yaml
appId: com.example.app
---
- launchApp: com.example.app
```

Each command is a YAML list entry corresponding to a Maestro command (e.g., `launchApp`, `tapOn`, `inputText`, `pressKey`, `stopApp`, `installApp`).

Prerequisites

1. **Maestro CLI** installed and in PATH (`curl -Ls https://get.maestro.mobile.dev | bash`)
2. **Android**: ADB in PATH, emulator or physical device connected
3. **iOS**: Xcode with Simulator running, or a physical device with Maestro driver installed

Constructor

```
adapter := userflow.NewMaestroCLIAdapter(userflow.MobileConfig{
    PackageName:  "com.example.app",
    DeviceSerial: "emulator-5554",   // Optional
})
```

- `PackageName` is the Android package name or iOS bundle ID, used as the Maestro `appId`
- `DeviceSerial` is optional; when set, it is passed to Maestro via `--device`

API Reference

IsDeviceAvailable

```
available, err := adapter.IsDeviceAvailable(ctx)
```

Runs `maestro devices` and parses the output. When a `DeviceSerial` is configured, looks for that specific serial in the output. Otherwise, looks for any line containing "Connected", "device", or "emulator".

InstallApp

```
err := adapter.InstallApp(ctx, "/path/to/app.apk")
```

Generates a flow with `installApp: /path/to/app.apk` and executes it.

LaunchApp, StopApp

```
err := adapter.LaunchApp(ctx)
err = adapter.StopApp(ctx)
```

Generate flows with `launchApp: <package>` and `stopApp: <package>` respectively.

IsAppRunning

```
running, err := adapter.IsAppRunning(ctx)
```

Runs `maestro hierarchy` to dump the current view hierarchy. Returns true if the output contains the configured package name. Returns false (without error) if the hierarchy command fails.

TakeScreenshot

```
png, err := adapter.TakeScreenshot(ctx)
```

Runs `maestro screenshot <path>` and reads the resulting PNG file from disk. The file is cleaned up after reading.

Tap

```
err := adapter.Tap(ctx, 540, 800)
```

Generates a flow with:

```
- tapOn:
    point: "540,800"
```

SendKeys

```
err := adapter.SendKeys(ctx, "hello world")
```

Generates a flow with `inputText: hello world`. The text is typed into the currently focused input element.

PressKey

```
err := adapter.PressKey(ctx, "back")
```

Generates a flow with `pressKey: back`. Maestro supports key names: `back`, `home`, `enter`, `tab`, `backspace`, `volume_up`, `volume_down`, `power`, `lock`.

WaitForApp

```
err := adapter.WaitForApp(ctx, 30*time.Second)
```

Polls `IsAppRunning` every 500ms until the app is detected or the timeout expires.

RunInstrumentedTests

```
result, err := adapter.RunInstrumentedTests(ctx,
        "com.example.LoginTest")
```

Returns `nil, nil`. Maestro uses its own YAML-based test format and does not support Android instrumented test runners (JUnit/Espresso). For instrumented tests, use the `EspressoAdapter` or `AppiumAdapter`.

Close, Available

```
err := adapter.Close(ctx)        // Removes temp directory
ok := adapter.Available(ctx)     // Runs "maestro --version"
```

Device Management

Maestro discovers devices automatically through ADB (Android) or Xcode (iOS). To target a specific device:

```
adapter := userflow.NewMaestroCLIAdapter(userflow.MobileConfig{
    PackageName:  "com.example.app",
    DeviceSerial: "emulator-5554",
})
```

The adapter passes `--device emulator-5554` to all `maestro test` invocations when `DeviceSerial` is set.

For multiple devices, create separate adapter instances:

```
phone := userflow.NewMaestroCLIAdapter(userflow.MobileConfig{
    PackageName: "com.example.app", DeviceSerial: "emulator-5554",
})
tablet := userflow.NewMaestroCLIAdapter(userflow.MobileConfig{
    PackageName: "com.example.app", DeviceSerial: "emulator-5556",
})
```

Limitations

Aspect	Detail
Instrumented tests	Not supported. Maestro is a UI-level testing tool only
Element selectors	No CSS/XPath selectors. Maestro uses text matching and accessibility IDs
Performance	Each method call spawns a full `maestro test` process

Aspect	Detail
Complex gestures	Limited to tap and text input. Swipe and drag require custom YAML
Return values	No way to extract element text or attributes programmatically
Cross-platform	Works on both Android and iOS, but YAML commands may differ per platform
YAML escaping	Values containing YAML special characters (`:`, `#`, `[`, `]`) are automatically quoted

When to Use

- Rapid prototyping of mobile test flows
- Teams already using Maestro for their mobile testing
- Simple UI verification flows (tap, type, assert visible)
- When you do not need element-level queries or data extraction

For more complex mobile testing (element queries, attribute inspection, instrumented tests), use `AppiumAdapter` or `EspressoAdapter` instead.

Integration with Challenge Templates

```
adapter := userflow.NewMaestroCLIAdapter(userflow.MobileConfig{
    PackageName: "com.example.app",
})

challenge := userflow.NewMobileFlowChallenge(
    "CH-MAESTRO-001",
    "Maestro Login Flow",
    "Verify mobile login via Maestro YAML flows",
    nil,
    adapter,
    userflow.MobileFlow{
        Name: "maestro-login",
        Steps: []userflow.MobileStep{
            {Name: "tap-email", Action: "tap", X: 540, Y: 800},
            {Name: "type-email", Action: "send_keys", Text:
         "user@test.com"},
            {Name: "tap-login", Action: "tap", X: 540, Y: 1100},
            {Name: "verify", Action: "screenshot"},
        },
    },
)
```

Source Files

- Interface: `pkg/userflow/adapter_mobile.go`
- Implementation: `pkg/userflow/maestro_adapter.go`